

Weekly Report

QCar Refactor & CARLA Simulation

Qi LI

July 14, 2026

Contents

1	Work summary	1
2	Platform refactor	1
2.1	Platform redesign	1
2.2	What I built	1
2.3	Multi-vehicle simulation	2
2.4	What is complete	2
2.5	Still to do	2
3	CARLA integration	2
4	Conclusion	3
4.1	Next steps	3

1 Work summary

- Vehicle lifecycle queries were separated from IO, estimation, planning, control, and V2V modules. Meanwhile, the platform is software-free and easy to extend.
- Carla support was added, and a v2v test is implemented.

2 Platform refactor

2.1 Platform redesign

The old platform put most vehicle work inside one large `VehicleLogic` class. This made small changes risky and made it hard to test a module on its own.

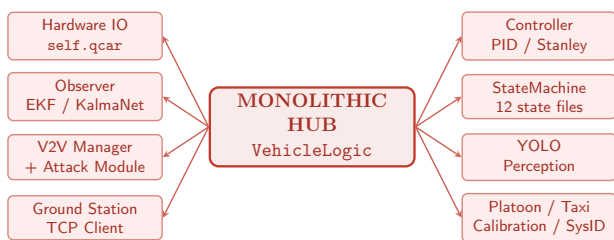


Figure 1: Before: one `VehicleLogic` class directly handled every subsystem.

1. **One large main class.** `VehicleLogic` handled the lifecycle, sensors, observer, state machine, con-

troller, V2V, telemetry, logging, and optional research features. [The main loop grew to more than 800 lines, so it was difficult to review.](#)

2. **Modules were tightly connected.** A change in one feature could break an unrelated feature.
3. **Data flow was mixed.** Changing sensor data or V2V data required edits in several modules.
4. **Configuration was scattered.** Selecting a backend or controller meant searching several files.
5. **Simulation and hardware were mixed.** Adding a simulator could require changing the core runtime.

The figure shows why the old structure was difficult to extend: the main class talks directly to every subsystem.

2.2 What I built

I moved the main lifecycle into a small `VehicleRuntime`. It starts, steps, and stops the vehicle. The state machine only decides whether driving is safe. IO, observer, planner, controller, and V2V are now separate modules built by the module factory.

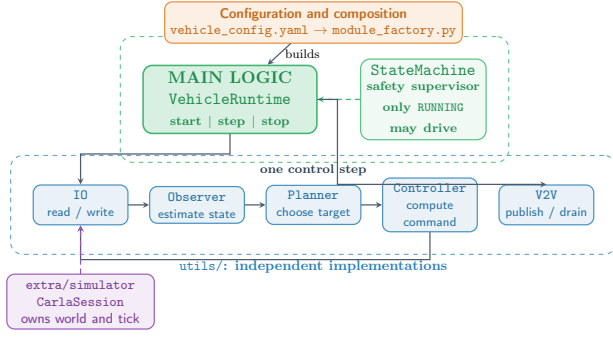


Figure 2: After: `core/` keeps the main logic; `utils/` holds replaceable modules.

2.3 Multi-vehicle simulation

Before, the scenario configuration need to be made manually by running the command with parsers. I added scenarios configuration and `VehicleProcessSpec` so each vehicle runs in its own process with its own `VehicleRuntime`. This keeps each control loop independent while allowing V2V communication. In CARLA, one process owns the world tick.

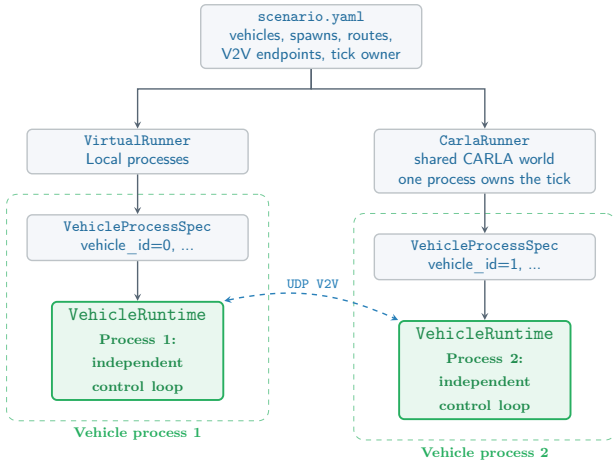


Figure 3: One scenario starts separate vehicle processes. V2V connects them.

2.4 What is complete

This is the current status of the refactor work.

Step	State	Key outcome
Configuration	Done	YAML profiles and validation
Contracts	Done	Shared dataclasses
Runtime	Done	Runtime coordinator and module factory
IO adapters	Done	QCar, virtual, and CARLA back-ends
CARLA	Done	Session, adapter, smoke/control tests
Multi-vehicle	Done	Process specs and YAML scenarios
V2V / fleet	Partial	UDP transport; fleet work pending
Ground station	Planned	Bridge and visual application pending

Table 1: Current implementation status.

2.5 Still to do

- **Fleet features:** peer membership, fleet snapshots, stale-peer removal, and payload decoding.
- **Ground-station bridge:** Vehicle-side TCP bridge with MessagePack framing, command queue, telemetry queue.
- **Ground-station application:** Visual telemetry display and command interface.
- **Physical QCar2:** the licence is expired, so I kept this code as reference only.
- **ROS adapters:** Deferred until external ROS transport contracts are defined.
- **Operation strategies:** Platoon, manual mode, taxi, and calibration as pluggable strategies behind the RUNNING state.

3 CARLA integration

I checked CARLA in stages. Unit tests use fake CARLA objects, so they run without a server. Then direct-run tests use a local CARLA server and save CSV files and plots for review.

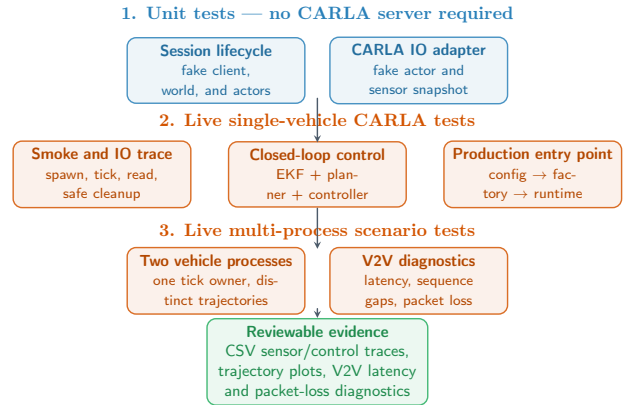


Figure 4: CARLA checks: fakes first, then live single-vehicle and multi-vehicle runs.

The first saved plot shows the closed-loop control run. It compares the planned path, GPS, EKF, speed, commands, and position error.

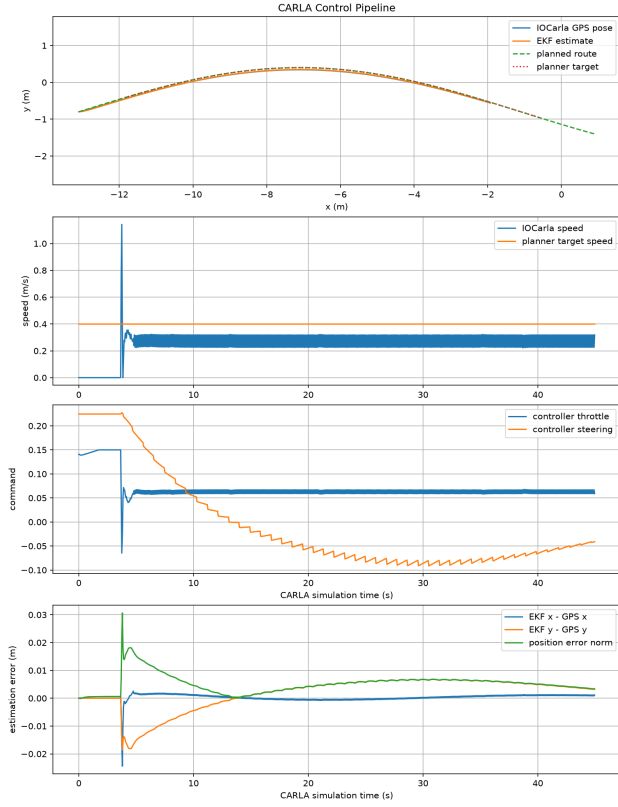


Figure 5: Closed-loop CARLA control result.

The multi-vehicle run also records the V2V path, latency, queue delay, and packet loss.



Figure 6: Two-vehicle CARLA V2V result.

The unit and integration tests are under `test/`. This gives me a quick way to find whether a prob-

lem comes from the algorithm, runtime, or simulator before running a larger scenario.

4 Conclusion

4.1 Next steps

- Finish fleet building and the ground-station path in the refactored structure.
Will be finished in this week.
- Test the refactored code on QCar2 hardware when the licence is available.
Will be finished in two weeks.
- Study 2D platoon control and build a distributed high-gain observer.
Will be finished in two weeks.
- Use MATLAB to test the observer before moving it into the vehicle platform.
Will be finished in two weeks.

The refactored platform is now running with CARLA. Next, I will finish the fleet and ground-station features and test the platform on QCar2 hardware.

I will first test the longitudinal fleet system. Then I will formulate the triangular form for the 2D fleet system and validate it in MATLAB. After that, I will test it in CARLA simulation, and finally on QCar2 hardware.

I am currently reading E.D. Sontag's tutorial on stability. After that, I will investigate whether an interval observer with more advanced design methods can provide better stability. This could be practical for the vehicle platform and also help me enter the observer-design research field.